AI Paradigm Comparator

Traditional vs Generative vs Agentic AI on the same query

Project 1 Report

Name: _____

Register No.: _____

Course: _____

Date: 4 November 2026

Live demo: https://huggingface.co/spaces/kabilash1/ai-paradigm-comparator

1. Problem Statement

Artificial intelligence systems can answer the same question in very different ways. This project builds one program that answers the same query with three AI paradigms and compares how each behaves:

- **Traditional AI:** exact rule matching against a dictionary of known questions.
- **Generative AI:** random combination of knowledge fragments, so the response differs on each run.
- **Agentic AI:** an iterative loop that searches a knowledge base, checks whether the evidence is sufficient, broadens the search when it is not, and synthesizes an answer.

The domain is AI & Machine Learning fundamentals. The system is tested with four queries, two that the rule base can answer and two that it cannot, and a comparison table records which approach succeeded on each. The core logic uses only the Python standard library (random, re, dataclasses); the web front end uses Streamlit.

2. Approach

The system uses an AI and machine learning knowledge domain with 9 exact-match rules and a 16-entry knowledge base, where each entry has searchable tags and a text fragment. Traditional AI normalizes the query (lower-case, punctuation removed) and looks it up in the rules dictionary, returning a fixed answer or "No match found". This makes it precise but brittle, since any rephrasing fails. Generative AI collects fragments relevant to the query, mixes in random distractors, and joins 2 –3 randomly chosen fragments with connectors. Its output therefore changes on every run and can drift off-topic, which models the inconsistency and hallucination risk of generative systems. Agentic AI runs an iterative loop: it searches the knowledge base with specific keywords, self-checks whether at least two matches were found, and if not, broadens its keywords using a synonym map and generic words, switching to fuzzy matching on the final iteration. Once the evidence is sufficient, it synthesizes an answer from the top three matches. Four test queries (two rule hits, two misses) and a comparison table show that only the agentic approach answers all queries reliably.

3. Design of Each Paradigm

3.1 Traditional AI: traditional_ai(query)

The query is normalized (lower-cased, punctuation removed, spaces collapsed) and looked up in `RULES`, a dictionary of 9 question → answer pairs. A hit returns the fixed answer; anything else returns exactly `"No match found"`. The behaviour is deterministic and precise, but any rephrasing such as "Explain deep learning" fails.

3.2 Generative AI: generative_ai(query)

1. Extract specific keywords (stopwords and generic words such as "learning" removed).
2. Build a pool: knowledge-base entries whose tags overlap the keywords, plus 2 random distractors.
3. Randomly sample 2 or 3 fragments and join them with random connectors ("Moreover,", "Also,", "In addition," ...), lower-casing the next fragment unless it starts with an acronym such as NLP.

4. Return `GenerativeResult(answer, on_topic)`, where on_topic is true if at least one relevant fragment was chosen.

Because both the fragment choice and the connectors are random, output varies across runs, and a response can drift off-topic (reported as WEAK). This models the fluency-without-guarantees of generative systems.

3.3 Agentic AI: agentic_ai(query, kb)

5. **Search:** iteration 1 uses the specific keywords with exact tag matching.
6. **Self-check:** the result is sufficient if at least MIN_MATCHES = 2 entries match.
7. **Broaden:** if not, add synonyms from the SYNONYMS map (e.g. deep → neural, layers, networks) and re-add the generic words. The final iteration (MAX_ITERATIONS = 3) switches to fuzzy substring matching.
8. **Synthesize:** entries are ranked by matches on the original keywords, then by total matches, and the top TOP_K = 3 are combined into the answer.

Every iteration is printed and stored in `AgentResult.log`. The result also carries the iteration count and a status: OK (≥ 2 matches), PARTIAL (1) or FAIL (0).

Constant	Value	Purpose
MIN_MATCHES	2	Matches needed for the self-check to pass
MAX_ITERATIONS	3	Upper bound on search rounds
TOP_K	3	Entries combined into the synthesized answer
GENERIC_WORDS	learning, model, models	Ignored in iteration 1, re-added when broadening

4. Knowledge Base

Traditional AI uses 9 rules: "what is machine learning", "what is overfitting", "what is a neural network", "what is supervised learning", "what is unsupervised learning", "what is reinforcement learning", "what is artificial intelligence", "what is natural language processing", "what is computer vision". Generative and Agentic AI share the 16-entry knowledge base below. The deep-learning entry is deliberately tagged only {deep, dl}, so the agent must broaden its search to find supporting entries.

ID	Tags	Text
ml-1	learning, machine, ml	Machine learning lets computers improve at a task by learning patterns from data.
ml-2	algorithms, machine, training	Machine learning algorithms are trained on examples and then tested on data they have never seen.
ml-3	applications, machine, prediction	Machine learning powers spam filters, recommendations and fraud detection.
sup-1	classification, labeled, supervised	Supervised learning uses labeled data to learn a mapping from inputs to known outputs.
uns-1	clustering, unlabeled, unsupervised	Unsupervised learning discovers clusters and structure in data without labels.
rl-1	agent, reinforcement, reward	Reinforcement learning teaches an agent by trial and error, using rewards and penalties.
nn-1	layers, network, networks, neural	A neural network stacks layers of artificial neurons that transform inputs step by step.
nn-2	backpropagation, networks, neural, weights	Neural networks learn by backpropagation, adjusting weights to reduce prediction error.
dl-1	deep, dl	Deep learning uses neural networks with many hidden layers to learn complex

ID	Tags	Text
		features automatically.
cnn-1	cnn, convolutional, image	CNNs use convolution filters to detect edges and shapes, which makes them ideal for images.
over-1	generalization, memorize, overfitting	Overfitting happens when a model memorizes the training set and fails to generalize to new data.
reg-1	dropout, penalty, regularization	Regularization methods such as dropout and weight decay reduce overfitting.
gd-1	descent, gradient, optimization	Gradient descent minimizes a loss function by repeatedly stepping against the gradient.
nlp-1	language, nlp, text	NLP lets machines process human language for translation, chatbots and sentiment analysis.
trf-1	attention, transformer, transformers	Transformers use self-attention to weigh the relevance of every word in a sequence.
cv-1	computer, recognition, vision	Computer vision teaches machines to recognize objects and scenes in images and video.

Table 1: Knowledge base (16 entries)

5. Full Source Code (paradigm_comparator.py)

The program passes pycodestyle (PEP-8, max line length 79) with no warnings.

```
"""AI Paradigm Comparator.

Answers the same query with three AI approaches and compares them:

1. Traditional AI - exact rule matching from a dictionary.
2. Generative AI - random combination of knowledge fragments.
3. Agentic AI    - iterative search, self-check and query broadening.

Topic: AI & Machine Learning fundamentals.
Only the Python standard library is used.
"""

import random
import re
from dataclasses import dataclass, field

# ======================================================================
# 1. CONSTANTS AND KNOWLEDGE SOURCES
# ======================================================================

NO_MATCH = "No match found"
MIN_MATCHES = 2      # matches the agent needs to call an answer sufficient
MAX_ITERATIONS = 3   # hard limit on agent search rounds
TOP_K = 3            # entries used to synthesize the final answer

# Traditional AI: normalized question -> fixed answer.
RULES = {
    "what is machine learning": (
        "Machine learning is a branch of AI where systems learn "
        "patterns from data instead of being explicitly programmed."
    ),
    "what is overfitting": (
        "Overfitting is when a model memorizes training data, noise "
        "included, and then performs poorly on new unseen data."
    ),
    "what is a neural network": (
        "A neural network is a model made of layers of connected "
        "neurons that learn weights to map inputs to outputs."
    ),
    "what is supervised learning": (
        "Supervised learning trains a model on labeled examples so it "
        "can predict labels for new inputs."
    ),
    "what is unsupervised learning": (
        "Unsupervised learning finds hidden structure, such as "
        "clusters, in data that has no labels."
    ),
    "what is reinforcement learning": (
        "Reinforcement learning trains an agent to act in an "
        "environment by maximizing cumulative reward."
    ),
    "what is artificial intelligence": (
        "Artificial intelligence is the field of building machines "
        "that perform tasks normally requiring human intelligence."
    ),
    "what is natural language processing": (
        "Natural language processing lets computers understand, "
        "interpret and generate human language."
```

```
    ),
    "what is computer vision": (
        "Computer vision enables machines to interpret and understand "
        "images and video."
    ),
}

# Knowledge base shared by the Generative and Agentic approaches.
KNOWLEDGE_BASE = [
    {"id": "ml-1", "tags": {"machine", "learning", "ml"},
     "text": "Machine learning lets computers improve at a task by "
             "learning patterns from data."},
    {"id": "ml-2", "tags": {"machine", "algorithms", "training"},
     "text": "Machine learning algorithms are trained on examples and "
             "then tested on data they have never seen."},
    {"id": "ml-3", "tags": {"machine", "applications", "prediction"},
     "text": "Machine learning powers spam filters, recommendations "
             "and fraud detection."},
    {"id": "sup-1", "tags": {"supervised", "labeled", "classification"},
     "text": "Supervised learning uses labeled data to learn a mapping "
             "from inputs to known outputs."},
    {"id": "uns-1", "tags": {"unsupervised", "clustering", "unlabeled"},
     "text": "Unsupervised learning discovers clusters and structure "
             "in data without labels."},
    {"id": "rl-1", "tags": {"reinforcement", "reward", "agent"},
     "text": "Reinforcement learning teaches an agent by trial and "
             "error, using rewards and penalties."},
    {"id": "nn-1", "tags": {"neural", "network", "networks", "layers"},
     "text": "A neural network stacks layers of artificial neurons that "
             "transform inputs step by step."},
    {"id": "nn-2",
     "tags": {"neural", "networks", "backpropagation", "weights"},
     "text": "Neural networks learn by backpropagation, adjusting "
             "weights to reduce prediction error."},
    {"id": "dl-1", "tags": {"deep", "dl"},
     "text": "Deep learning uses neural networks with many hidden "
             "layers to learn complex features automatically."},
    {"id": "cnn-1", "tags": {"cnn", "convolutional", "image"},
     "text": "CNNs use convolution filters to detect edges and shapes, "
             "which makes them ideal for images."},
    {"id": "over-1",
     "tags": {"overfitting", "generalization", "memorize"},
     "text": "Overfitting happens when a model memorizes the training "
             "set and fails to generalize to new data."},
    {"id": "reg-1", "tags": {"regularization", "dropout", "penalty"},
     "text": "Regularization methods such as dropout and weight decay "
             "reduce overfitting."},
    {"id": "gd-1", "tags": {"gradient", "descent", "optimization"},
     "text": "Gradient descent minimizes a loss function by repeatedly "
             "stepping against the gradient."},
    {"id": "nlp-1", "tags": {"nlp", "language", "text"},
     "text": "NLP lets machines process human language for translation, "
             "chatbots and sentiment analysis."},
    {"id": "trf-1", "tags": {"transformer", "transformers", "attention"},
     "text": "Transformers use self-attention to weigh the relevance "
             "of every word in a sequence."},
    {"id": "cv-1", "tags": {"vision", "computer", "recognition"},
     "text": "Computer vision teaches machines to recognize objects "
             "and scenes in images and video."},
]

STOPWORDS = {
    "what", "is", "a", "an", "the", "how", "do", "does", "explain",
    "from", "of", "to", "in", "on", "are", "about", "me", "tell",
    "why", "can", "and", "or", "with", "for", "give", "describe",
}

# Words too common to be useful in the agent's first, specific search.
GENERIC_WORDS = {"learning", "model", "models"}

# Query expansion used when the agent broadens its search.
SYNONYMS = {
    "deep": ["neural", "layers", "networks"],
    "overfitting": ["generalization", "regularization", "dropout"],
    "neural": ["network", "networks", "layers"],
    "networks": ["neural", "layers"],
    "learn": ["training", "weights", "backpropagation"],
    "data": ["training", "labeled"],
    "language": ["nlp", "text"],
    "vision": ["image", "cnn"],
    "image": ["vision", "cnn"],
    "gradient": ["optimization", "descent"],
}

# 2 queries Traditional AI answers (exact rules), 2 it cannot.
TEST_QUERIES = [
    "What is machine learning?",                # rule HIT
    "What is overfitting?",                     # rule HIT
    "Explain deep learning",                    # rule MISS
    "How do neural networks learn from data?",  # rule MISS
]


# ======================================================================
# 2. RESULT CONTAINERS
# ======================================================================
```

```
@dataclass
class GenerativeResult:
    """Answer from the Generative AI plus an on-topic flag."""

    answer: str
    on_topic: bool


@dataclass
class AgentResult:
    """Answer from the Agentic AI with its reasoning trace."""

    answer: str
    iterations: int
    status: str                       # "OK", "PARTIAL" or "FAIL"
    log: list = field(default_factory=list)


# ======================================================================
# 3. TEXT HELPERS
# ======================================================================


def normalize(text):
    """Lower-case text and strip punctuation and extra spaces."""
    text = re.sub(r"[^a-z0-9\s]", "", text.lower())
    return " ".join(text.split())


def extract_keywords(query):
    """Return query words with stopwords removed (order kept)."""
    words = normalize(query).split()
    return [w for w in words if w not in STOPWORDS]


def specific_keywords(query):
    """Return keywords without generic words (all if none remain)."""
    words = extract_keywords(query)
    return [w for w in words if w not in GENERIC_WORDS] or words


def _unique(words):
    """Remove duplicates from a list while keeping the order."""
    return list(dict.fromkeys(words))


# ======================================================================
# 4. TRADITIONAL AI - EXACT RULE MATCHING
# ======================================================================


def traditional_ai(query):
    """Look the normalized query up in RULES, else 'No match found'."""
    return RULES.get(normalize(query), NO_MATCH)


# ======================================================================
# 5. GENERATIVE AI - RANDOM FRAGMENT COMBINATION
# ======================================================================

CONNECTORS = [
    "Moreover,", "Also,", "In addition,", "Furthermore,",
    "Additionally,", "Notably,",
]


def _join_fragment(text):
    """Lower-case the first letter unless it starts an acronym."""
    first_word = text.split()[0]
    if len(first_word) > 1 and first_word[:2].isupper():
        return text
    return text[0].lower() + text[1:]


def generative_ai(query):
    """Randomly pick and join 2-3 fragments, so output varies."""
    keywords = set(specific_keywords(query))
    relevant = [e for e in KNOWLEDGE_BASE if e["tags"] & keywords]
    others = [e for e in KNOWLEDGE_BASE if e not in relevant]
    pool = relevant + random.sample(others, min(2, len(others)))
    # An off-topic query has no relevant entries, so the pool can be
    # smaller than 3: never ask for more fragments than it holds.
    size = min(random.choice([2, 3]), len(pool))
    chosen = random.sample(pool, size)
    parts = [chosen[0]["text"]]
    for entry in chosen[1:]:
        connector = random.choice(CONNECTORS)
        parts.append(f"{connector} {_join_fragment(entry['text'])}")
    on_topic = any(e in relevant for e in chosen)
    return GenerativeResult(" ".join(parts), on_topic)


# ======================================================================
# 6. AGENTIC AI - ITERATIVE SEARCH WITH SELF-CHECK
# ======================================================================
```

```
def _score(entry, keywords, fuzzy):
    """Count keywords matching an entry (exact tags or substrings)."""
    score = 0
    for word in keywords:
        if word in entry["tags"]:
            score += 1
        elif fuzzy and (
            word in entry["text"].lower()
            or any(len(t) > 2 and (word in t or t in word)
                   for t in entry["tags"])
        ):
            score += 1
    return score


def _search(kb, keywords, fuzzy, core):
    """Return entries with score > 0, best match first.

    Entries matching the original specific keywords (``core``) rank
    above entries that only match broadened ones.
    """
    scored = []
    for entry in kb:
        score = _score(entry, keywords, fuzzy)
        if score > 0:
            rank = (_score(entry, core, fuzzy), score)
            scored.append((rank, entry))
    scored.sort(key=lambda pair: pair[0], reverse=True)
    return [e for _, e in scored]


def _broaden(keywords, base_keywords):
    """Add synonyms and re-add the generic words that were dropped."""
    wider = list(keywords) + [w for w in base_keywords
                              if w in GENERIC_WORDS]
    for word in base_keywords:
        wider.extend(SYNONYMS.get(word, []))
    return _unique(wider)


def agentic_ai(query, kb=KNOWLEDGE_BASE, verbose=False):
    """Search the KB, self-check, broaden if needed, then synthesize."""
    base = extract_keywords(query)
    core = specific_keywords(query)
    keywords = list(core)
    log = []
    matches = []
    iteration = 0

    def say(message):
        """Record a trace line and print it when verbose."""
        log.append(message)
        if verbose:
            print(message)

    while iteration < MAX_ITERATIONS:
        iteration += 1
        fuzzy = iteration == MAX_ITERATIONS
        mode = "fuzzy" if fuzzy else "exact"
        say(f"Iteration {iteration}: searching {keywords} ({mode})")
        matches = _search(kb, keywords, fuzzy, core)
        count = len(matches)
        noun = "match" if count == 1 else "matches"
        if count >= MIN_MATCHES:
            say(f"  -> Found {count} {noun}. Sufficient!")
            break
        if iteration < MAX_ITERATIONS:
            say(f"  -> Found {count} {noun}. Not enough, broadening...")
            keywords = _broaden(keywords, base)
        else:
            say(f"  -> Found {count} {noun}. Out of iterations.")

    if len(matches) >= MIN_MATCHES:
        status = "OK"
    elif matches:
        status = "PARTIAL"
    else:
        status = "FAIL"
    top = matches[:TOP_K]
    answer = " ".join(e["text"] for e in top) if top else NO_MATCH
    return AgentResult(answer, iteration, status, log)


# ============================================================================
# 7. TEST HARNESS
# ============================================================================


def run_query(query, verbose=True):
    """Run one query through all three approaches.

    Prints each answer (and the agent trace) when ``verbose`` is True
    and returns a comparison row as a dict.
    """
    trad = traditional_ai(query)
    gen = generative_ai(query)
```

```
    if verbose:
        print("=" * 70)
        print(f"Query: '{query}'")
        print(f"[Traditional AI] {trad}")
        print(f"[Generative AI] {gen.answer}")
        print("[Agentic AI]")
    agent = agentic_ai(query, verbose=verbose)
    if verbose:
        print(f"Synthesized: {agent.answer}")
        print()
    return {
        "query": query,
        "traditional": "HIT" if trad != NO_MATCH else "MISS",
        "generative": "OK" if gen.on_topic else "WEAK",
        "agentic": f"{agent.status} ({agent.iterations} iter)",
    }


def show_generative_variation(query="What is machine learning?",
                              runs=3):
    """Run Generative AI several times to show output variation."""
    print(f"Generative AI variation for '{query}':")
    for number in range(1, runs + 1):
        print(f"  Run {number}: {generative_ai(query).answer}")
    print()


def print_comparison(rows=None):
    """Print a fixed-width table of which approach succeeded.

    ``rows`` come from run_query(); if omitted the test queries are
    run silently first.
    """
    if rows is None:
        rows = [run_query(q, verbose=False) for q in TEST_QUERIES]
    header = f"{'Query':<42}| {'Traditional':<12}| " \
             f"{'Generative':<11}| Agentic"
    print(header)
    print("-" * len(header))
    for row in rows:
        print(f"{row['query']:<42}| {row['traditional']:<12}| "
              f"{row['generative']:<11}| {row['agentic']}")
    print()


# ======================================================================
# 8. MAIN PROGRAM
# ======================================================================


def main():
    """Run every demo section of the assignment."""
    print("AI PARADIGM COMPARATOR")
    print(f"Rules: {len(RULES)} | Knowledge base: "
          f"{len(KNOWLEDGE_BASE)} entries\n")
    rows = [run_query(query) for query in TEST_QUERIES]
    show_generative_variation()
    print("COMPARISON TABLE")
    print_comparison(rows)


if __name__ == "__main__":
    main()
```

6. Console Output

Output of `python paradigm_comparator.py` (saved as console_output.txt). Generative AI lines differ on every run.

```
AI PARADIGM COMPARATOR
Rules: 9 | Knowledge base: 16 entries

======================================================================
Query: 'What is machine learning?'
[Traditional AI] Machine learning is a branch of AI where systems learn patterns from data instead
    of being explicitly programmed.
[Generative AI] Machine learning powers spam filters, recommendations and fraud detection. Moreover,
    regularization methods such as dropout and weight decay reduce overfitting.
[Agentic AI]
Iteration 1: searching ['machine'] (exact)
  -> Found 3 matches. Sufficient!
Synthesized: Machine learning lets computers improve at a task by learning patterns from data.
    Machine learning algorithms are trained on examples and then tested on data they have never
    seen. Machine learning powers spam filters, recommendations and fraud detection.

======================================================================
Query: 'What is overfitting?'
[Traditional AI] Overfitting is when a model memorizes training data, noise included, and then
    performs poorly on new unseen data.
[Generative AI] Transformers use self-attention to weigh the relevance of every word in a sequence.
    Moreover, gradient descent minimizes a loss function by repeatedly stepping against the
```

```
    gradient.
[Agentic AI]
Iteration 1: searching ['overfitting'] (exact)
  -> Found 1 match. Not enough, broadening...
Iteration 2: searching ['overfitting', 'generalization', 'regularization', 'dropout'] (exact)
  -> Found 2 matches. Sufficient!
Synthesized: Overfitting happens when a model memorizes the training set and fails to generalize to
    new data. Regularization methods such as dropout and weight decay reduce overfitting.

======================================================================
Query: 'Explain deep learning'
[Traditional AI] No match found
[Generative AI] Machine learning powers spam filters, recommendations and fraud detection.
    Furthermore, deep learning uses neural networks with many hidden layers to learn complex
    features automatically.
[Agentic AI]
Iteration 1: searching ['deep'] (exact)
  -> Found 1 match. Not enough, broadening...
Iteration 2: searching ['deep', 'learning', 'neural', 'layers', 'networks'] (exact)
  -> Found 4 matches. Sufficient!
Synthesized: Deep learning uses neural networks with many hidden layers to learn complex features
    automatically. A neural network stacks layers of artificial neurons that transform inputs step
    by step. Neural networks learn by backpropagation, adjusting weights to reduce prediction error.

======================================================================
Query: 'How do neural networks learn from data?'
[Traditional AI] No match found
[Generative AI] A neural network stacks layers of artificial neurons that transform inputs step by
    step. Furthermore, neural networks learn by backpropagation, adjusting weights to reduce
    prediction error. Also, computer vision teaches machines to recognize objects and scenes in
    images and video.
[Agentic AI]
Iteration 1: searching ['neural', 'networks', 'learn', 'data'] (exact)
  -> Found 2 matches. Sufficient!
Synthesized: A neural network stacks layers of artificial neurons that transform inputs step by
    step. Neural networks learn by backpropagation, adjusting weights to reduce prediction error.

Generative AI variation for 'What is machine learning?':
  Run 1: Machine learning lets computers improve at a task by learning patterns from data. Moreover,
    machine learning powers spam filters, recommendations and fraud detection. In addition,
    unsupervised learning discovers clusters and structure in data without labels.
  Run 2: Machine learning algorithms are trained on examples and then tested on data they have never
    seen. Notably, deep learning uses neural networks with many hidden layers to learn complex
    features automatically. Moreover, machine learning powers spam filters, recommendations and
    fraud detection.
  Run 3: Machine learning algorithms are trained on examples and then tested on data they have never
    seen. In addition, machine learning powers spam filters, recommendations and fraud detection.

COMPARISON TABLE
Query                                | Traditional | Generative | Agentic
----------------------------------------------------------------------------
What is machine learning?            | HIT         | OK         | OK (1 iter)
What is overfitting?                 | HIT         | WEAK       | OK (2 iter)
Explain deep learning                | MISS        | OK         | OK (2 iter)
How do neural networks learn from data?  | MISS        | OK         | OK (1 iter)
```

7. Comparison and Discussion

Query	Traditional	Generative	Agentic
What is machine learning?	HIT	OK	OK (1 iter)
What is overfitting?	HIT	WEAK	OK (2 iter)
Explain deep learning	MISS	OK	OK (2 iter)
How do neural networks learn from data?	MISS	OK	OK (1 iter)

Table 2: Which approach succeeded on each test query (from the console run above)

- **Traditional AI** answered only the two queries that match a rule word for word (HIT) and returned "No match found" for the rephrased ones. It is exact and repeatable but brittle.
- **Generative AI** always produces fluent text, but which fragments appear is random. In this run, the "overfitting" answer was rated WEAK because neither chosen fragment was tagged with the query keyword; on other runs the same query is rated OK. The variability check shows three different answers to one query.
- **Agentic AI** answered all four queries. For "What is overfitting?" and "Explain deep learning" its first search found only one match, so the self-check failed; it broadened with synonyms and succeeded on iteration 2. Its answer is grounded in the knowledge base and its reasoning trace is visible.

The trade-off is cost: the agent performs several searches per query, while the rule lookup is a single dictionary access. For known questions, rules are cheapest; for open-ended questions, the agentic loop is far more robust, and the generative approach is the least reliable.

8. Deployment

The Streamlit app (app.py) imports the three functions from paradigm_comparator.py, so no logic is duplicated. It shows the three answers side by side with HIT/MISS, OK/WEAK and status badges, a Regenerate button, the agent's reasoning trace and the comparison table.

Hugging Face Spaces now hosts Streamlit servers only on paid hardware, so the app is deployed as a free static Space. index.html uses stlite to run the unchanged app.py in the visitor's browser through Pyodide (Python compiled to WebAssembly). Public URL: https://huggingface.co/spaces/kabilash1/ai-paradigm-comparator

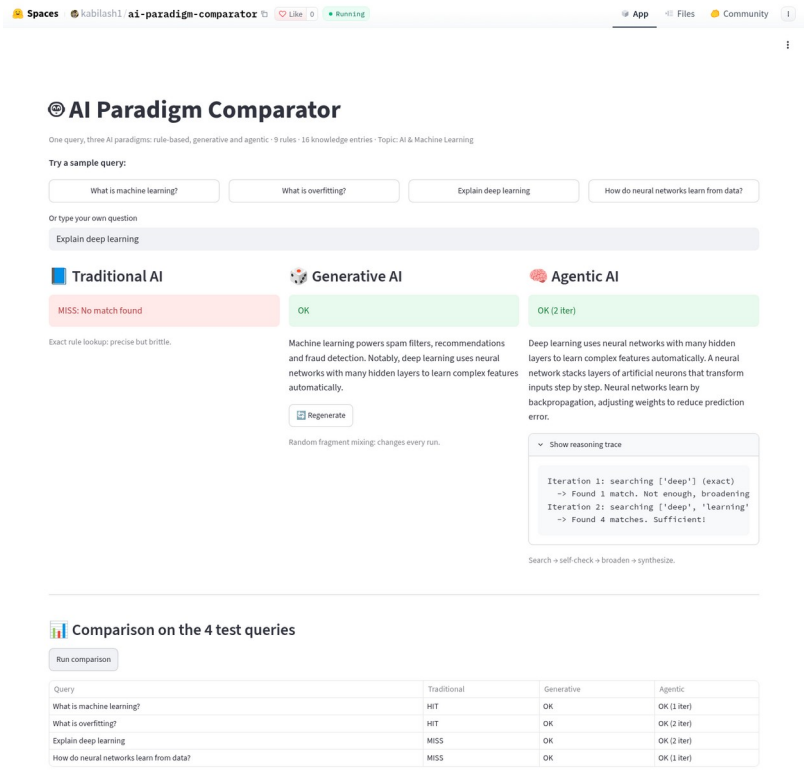


Figure 1: The deployed app on Hugging Face Spaces after the query "Explain deep learning" and Run comparison

9. Conclusion

The project implements the three paradigms on one shared knowledge domain and shows their distinct behaviour on the same queries. Traditional AI is precise but cannot handle unseen phrasing. Generative AI always responds but is inconsistent and sometimes off-topic. Agentic AI combines search, a self-check and query expansion to answer every test query, at the cost of extra iterations. Together they illustrate why modern AI systems increasingly wrap generative models in agentic loops that check and ground their output.